The Language of Numbers

Initialize_Probability: From Discrete Counts to Dense Vectors... [START]

The Language of Numbers: Data as Fuel

The Mechanical Foundation

> As we learned in the previous lecture, Neural Networks learn by adjusting **weights** to minimize error.

> They are the “engines” of AI, providing the structure for learning.

> **The Problem:** An engine with no fuel (input) cannot produce any work (predictions).

The Information Fuel (Data)

> For any language model, the fuel is **Textual Data.**

> Models process millions of sentences to find patterns, frequencies, and relationships.

> **The Constraint:** Computers only understand numbers. They cannot read strings like “Apple”; they only process floating-point numbers.

The Mathematical Bridge

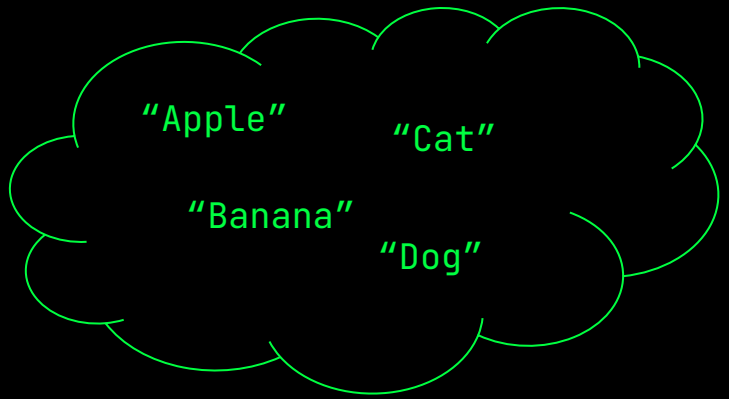
> To fuel a Neural Network with language, we must translate words into a numerical format.

> This chapter explores the **Math of Meaning** → converting words into mathematical **addresses** (vectors).

Words are for Humans. Numbers are for Machines.

The Messy World of Language

> **Content:** Raw strings, characters, and sentences.



Mathematical DNA

> **Content:** Vectors and Tensors.

[1,0,0,0], [0.12, -0.45, 0.88], [0.3, 0.14, -0.42, 0.11]...

The Probability of a Sentence: AI’s Statistical Roots

Before we had “Meaning,” we had Frequency

> **N-Grams:** A N-gram is a contiguous sequence of *N* words from a given text. It treats language as a game of chance, asking: “Statistically, how often does Word B follow Word A?”

> **Example:** “I went to the bank today.”
> > **Unigram (1-gram):** “I”, “went”, “to”, “the”, “bank”, “today.”
> > **Bigram (2-gram):** “I went”, “went to”, “to the”, “the bank”, “bank today.”

> **The Goal:** By counting these occurrences in a large corpus, we estimate the probability of the next word.

This was the era of the “best guess.”

We didn’t care what a “bank” was; we only cared that the word “river” or “money” appeared **frequently** in nearby data.

The N-gram Hierarchy: Levels of Context

The more you look back. The better you predict.

Unigram (N=1): The Bag of Words

Logic → Each word is considered independently; order is ignored

Example → “I went to the bank.” = “The bank I went to.”

Bigram (N=2): The Neighbor Check

Logic → Predicts a word based solely on the one before it.

Example → Knows “New” is likely followed by “York” rather than “Apple”

Trigram (N=3): The Local Context

Logic → Predicts a word based on the previous two words.

Example → Knows “Hot and” is likely followed by “Cold”

Note: While increasing N improves accuracy, it creates a “combinatorial explosion”. A 5-gram model for a 50,000-word vocabulary would require tracking more combinations that there are atoms in the solar system.

The Markov Assumption: The Math of Chance

> **The Concept:** N-grams rely on the **Markov Assumption**, the idea that we can predict the future based solely on the present, without needing to consider the entire past.

> **The Bigram Calculation:** The probability of a sentence like “I love dogs” is found by multiplying individual probabilities:

$$P(\text{I love dogs}) = P(\text{I}) \times P(\text{love}|\text{I}) \times P(\text{dogs}|\text{love})$$

> **The Statistical Guess:** To find P(love|I), the model counts occurrences in a large corpus:

$$P(\text{love}|\text{I}) = \frac{\text{Count}(\text{I love})}{\text{Count}(\text{I})}$$

This simplifies language into a manageable math problem. By looking at a large corpus (like Wikipedia), the model calculates the “best guess” based on historical patterns.

The Limitations: Why N-grams Hit a Wall

Three Main Bottlenecks

The Sparsity Problem

> As *N* increases, combinations explode.

The “Long-Range” Blind Spot:

> A 3-gram only looks at the previous two words; it has no long-term memory.

> It cannot connect a subject at the start of a long sentence to a verb at the end

No Concept of Similarity

> To an N-gram, “Cat” and “Kitten” are entirely different strings.

> If the model hasn’t seen the exact sequence “The kitten sat”, it assigns it a probability of 0.

N-grams treat words as isolated strings, not concepts.

They lack the “Math of Meaning” required to understand that similar words should have similar behaviors.

Scenario Question

You are training a Bigram (N=2) language model on a small corpus of text to predict the next word in a sentence. After processing the data you have the following counts:

> Count (“I”): 100
> Count (“I love”): 20
> Count (“love”): 50
> Count (“love dogs”): 10

According to the **Markov Assumption** and the formula for conditional probability, what is the probability P(love|I) that the model will predict “love” given the previous word is “I”?

[A] 0.05

[B] 0.1

[C] 0.2

[D] 0.5

Correct Answer

[C] is Correct!

Why?

The Logic: Based on the Markov Assumption, the bigram model calculates the “best guess” by dividing the number of times the sequence appeared by the total number of times the prefix appeared.

The Formula: P(love|I) = Count(I love) / Count(I)

The Calculation: 20/100 = 0.2

One-Hot Encoding: The Identity Vector

Giving Every Word a Unique Address

> **The Concept:** To turn strings into numbers, we represent each word as a vector the same size as our vocabulary (V)

> **The Method:** For each word, place a “1” in its unique position and “0”s everywhere else.

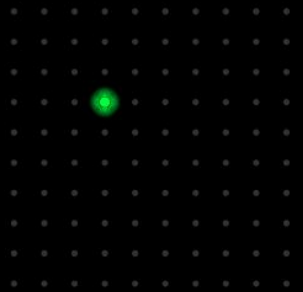
Example: [“Dog”, “Cat”, “Pig”, “Cow”]

> **Dog:** [1, 0, 0, 0]

> **Cat:** [0, 1, 0, 0]

> **Pig:** [0, 0, 1, 0]

> **Cow:** [0, 0, 0, 1]



Scaling Up:

In our 4-word example, we are in a 4-dimensional space. In a real LLM with a 50,000-word vocabulary, every word is a vector with 50,000 dimensions (mostly filled with 0s).

The Lookup Toggle: Why Computers Use One-Hot

The Matrix “Toggle”

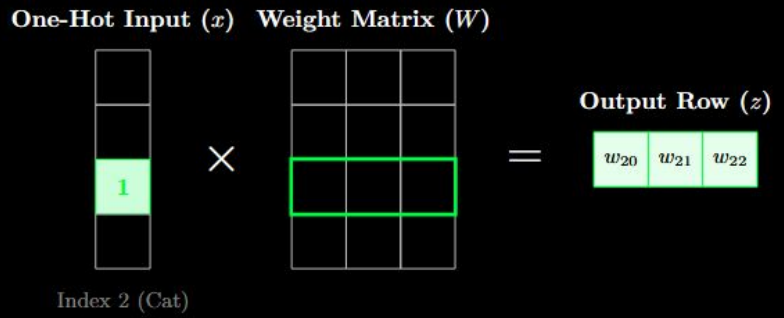
> **The Concept:** One-hot encoding is more than an ID; it acts as a **Lookup Toggle** when interacting with a neural network’s weights.

> **The Mechanism:** When you multiply a weight matrix by a one-hot vector, the math “toggles on” the specific row associated with that word and ignores the rest.

> **Efficiency:** The model learns specific rules for a word without those rules “leaking” into other words. If a model learns a weight for “Dog” in dimension 1, it won’t affect “Cat” in dimension 2.

In Practice

> By turning words into vectors, we allows neural networks to user their “Weights” to transform that word into a numerical representation with meaning. It is the first step in moving from **raw text** to **learned patterns**.



The Meaning Gap: A World Without Neighbors

The Problem of Orthogonality

> **The Concept:** In geometry, if two vectors are at a 90-degree angle, they are **Orthogonal**, meaning they have no mathematical relationship.

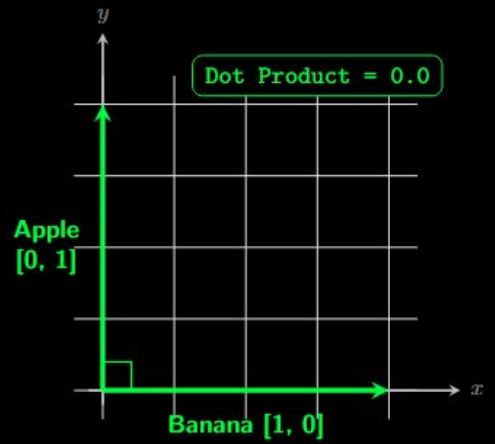
> **The Math:** In one-hot encoding, the "dot product" (a measure of similarity) between any two different words is always zero:

Apple * Banana = 0
Apple * Spaceship = 0

> **The Result:** The computer thinks "Apple" is just as related to Banana as it is to Spaceship

> **The Barrier:** There is no concept of synonyms, categories, or hierarchy → just empty space between IDs.

One-encoding is a significant leap forward because it gives every word a unique coordinate. However, it fails to capture meaning because words share no common dimensions. We need a "map" where where similar words live close together.



Distributional Semantics: The Company We Keep

Meaning is Found in the Neighborhood

> **The Foundation:** “You shall know a word by the company it keeps” - J.R. Firsth (1957)

> **The Logic:** Meaning is not found in a dictionary, but in the distribution of where a word usually appears.

> **The Comparison Test:**

“I will drink a hot cup of **coffee** this morning.”
“I will drink a hot cup of **tea** this morning.”

> **The Mathematical Inference:** Because “coffee” and “tea” share similar neighbors (“hot”, “cup”, “drink”), a model learns they must be similar types of things

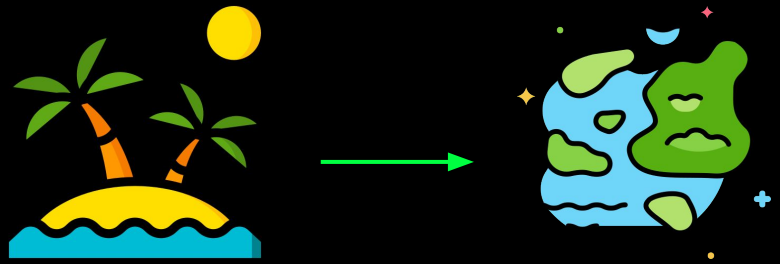
Example: If you read: “The **Squirgle** is a tool used for navigation in the ocean,” you instantly map “Squirgle” to *nautical tools* based on its neighbors. This is exactly how an LLM learns: it maps the statistical neighborhood until it understands context.

From Statistics to Geometry: Clustering Meaning

Moving Islands into Archipelagos

> **The Shift:** In the N-gram era, we used “company” only to predict the next word. In the modern era, we use it to calculate a word’s physical **location**.

> **The Mechanism:** When a model is trained on millions of sentences, the weights are updated based on contextual patterns



The Steps of Alignment

> **Observation:** The model sees “Coffee” and “Tea” appearing with the same neighbors (“hot”, “cup”, “drink”).

> **Update:** The model updates their vectors in a similar mathematical direction.

> **Convergence:** Over time, the isolated islands of these words are pulled together until they become **Semantically Similar** in the eyes of the computer.

Question: How can we move “Coffee” closer to “Tea” if their dimensions never overlap?

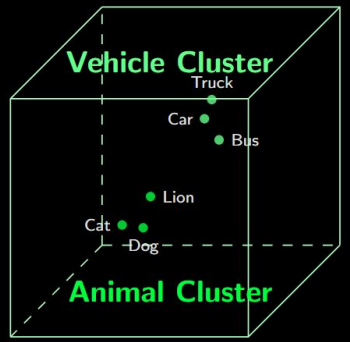
The Geography of Ideas: Dense Embeddings

The Geometry of Meaning

> **The Transition:** We move from **Sparse** One-Hot vectors to **Dense** Embeddings where every number is a non-zero value.

> **The Map: Distance = Difference →** The further apart two words are, the more different they are in meaning.

> **The Efficiency Leap:** For a model with a vocabulary with 50,000 words a **Dense Embedding** will probably be a short vector (384-4096 dimensions) packed with essentials as opposed to a 50,000-long vector full of mostly zeros in a sparse representation (wasteful memory).



The Clustering Box:

By forcing the model to squeeze all of human language into smaller, “Dense” space, we force it to find the common threads and patterns that connect words together. Words no longer live on isolated islands; they live in a structured world of neighbors.

Emergent Features: The Self-Taught Map

The Logic of Word2Vec

> **The Misconception:** Humans engineers do not label dimension (e.g., Dim 1 is size, Dim 2 is color...). In reality, meaning is an **emergent property** found by the model during training.

> **Predictive Alignment:** Word2Vec transforms words from random initial locations into a meaningful map by repeatedly using **Gradient Descent** to "pull" vectors toward the mathematical coordinates of their frequent neighbors and "push" them away from unrelated contexts.

The Training Cycle

> **Random Initialization:** Every word starts at a random location.

> **The Prediction Task:** The model is given a sentence with a missing word (e.g., "I drank __ this morning").

> **Gradient Descent:** If the model guesses "Hammer" instead of "Coffee," the loss function penalizes the mistake.

> **The Gravitational Pull:** Weights are updated to pull the vector for "Coffee" closer to its context neighbors and push "Hammer" further away.

Vector Arithmetic: Concepts as Math

Logic is Found in the Geometry

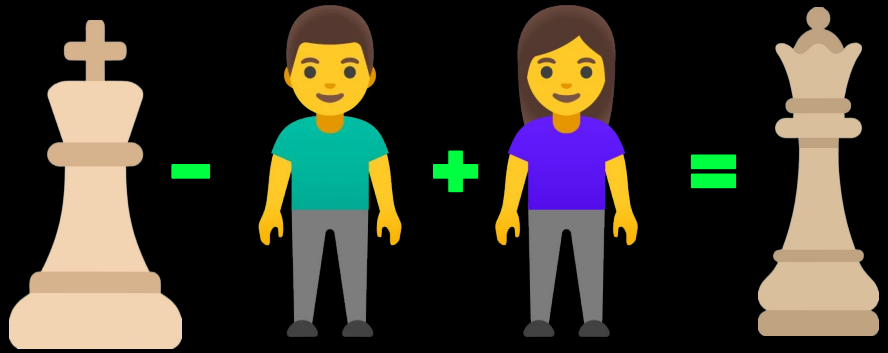
> **The Discovery:** Because embeddings encode meaning into specific dimensions, we can perform standard algebraic operations on abstract ideas.

> **The Famous Proof:**
>> Take the vector for **King** (Royalty + Male)
>> Subtract **Man** (Removes the 'Male' feature)
>> Add **Woman** (Adds the 'Female' feature)
>> **Result:** The math lands on a coordinate nearly identical to **Queen**.

> **Why it Works:** Dimensions are not random; they represent latent attributes like Gender, Size, and Sentiment that the model learned through scale.

The Shift from the N-gram logic:

Vector Arithmetic proves that the model hasn't just "memorized" word counts → it has built a logical representation of the world. The distance and direction between words represent real-world relationships.



Measuring Similarity: Angles over Distance

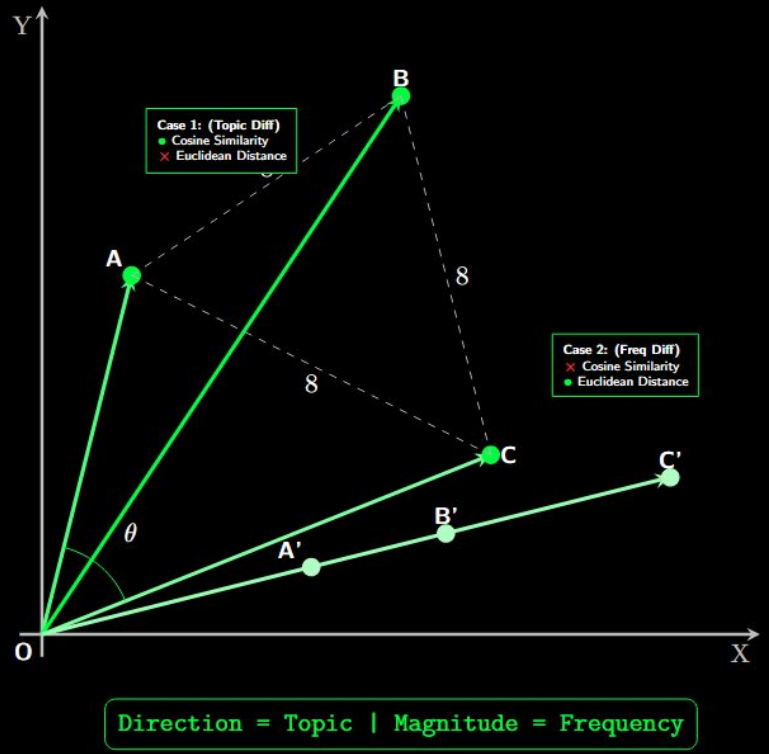
Why we Measure the Angle, Not the Gap

> **The Goal:** In high-dimensional space, we need to answer: "How similar are these two concepts?"

> **The Choice:** We use **Cosine Similarity** (measuring the angle between vectors) instead of simple straight-line Euclidean distance.

> **The Reason: Direction = Topic →** The angle tells us if the words share the same meaning or topic. **Gap = Frequency →** The length of a vector often just represents how common a word is in the training data.

> **The Advantage:** We want "Apple" and "iPhone" to be mathematically similar because of their topic, even if one word appears millions of times more than the other.



The Similarity Formula: The Math of Angles

The Formula Breakdown

> **The Formula:**

$$\text{Similarity}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

> The formula takes two massive lists of numbers and boils them down into a single score between -1 and 1 where:

1.0: Perfect synonyms (same direction)
0.0: Totally unrelated (Orthogonal)
-1.0: Perfect antonyms (opposite direction)

The Dot Product (A * B):

> Measures how much the two vectors point in the same direction

> High values indicate overlapping features; zero indicates no relationship

Normalization (||A|| ||B||):

> This is the “Equality Filter.” It divides by the length (magnitude) of the vectors.

> **The Result:** It cancels out the “frequency” of the word, leaving only the intent (semantics).

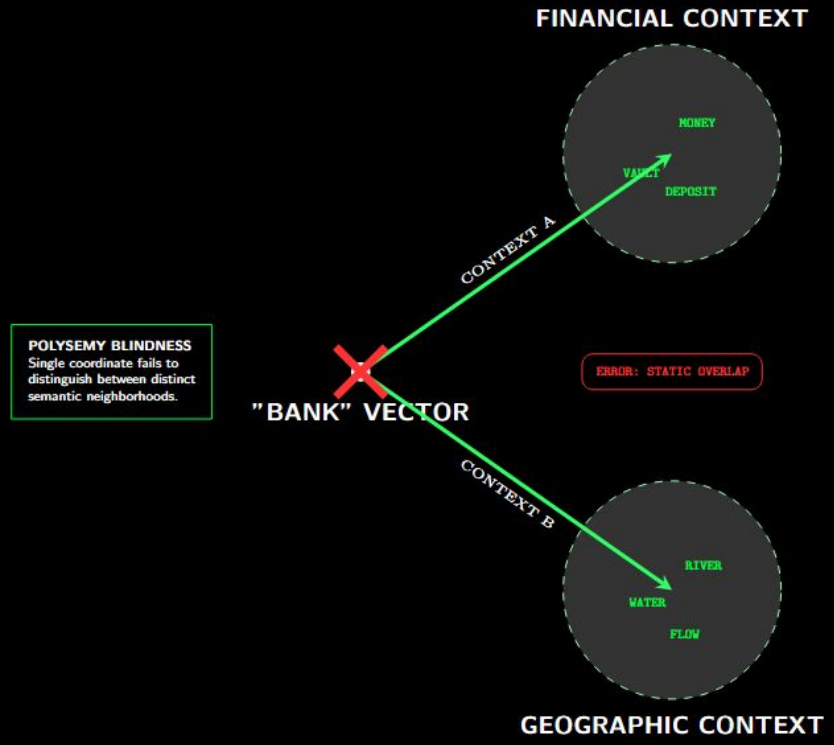
The Polysemy Problem: One Vector, Too Many Meanings

The Constraint of Static Embeddings

> **Definition:** Traditional models (like Word2Vec) assign each word **one unique vector** regardless of context.
> **The Problem:** Many words are **polysemous**, meaning they have multiple distinct meanings.
> **Confusion:** The model attempts to average all meanings into a single coordinate, creating a "meaning blur."

The "Bank" Scenario

> **Context A:** "I went to the **bank** to deposit money." (Financial)
> **Context B:** "I sat on the river **bank** to fish." (Geographic)
> **The Result:** To a static model, the word "bank" has the same address in both sentences, failing to distinguish between a river and a financial institution.



Scenario: The Semantic Search Optimization

You are building a search engine for a library. You have two models to choose from for your retrieval system: **Model Alpha**, which uses **One-Hot Encoding**, and **Model Beta**, which uses **Dense Embeddings**.

A user searches for the word “**Cat**”. The library database contains no books with the exact title “Cat”, but it has thousands of books titled “**Kitten**”.

Question:

Based on the material we’ve covered, which model will return the “Kitten” books to the user, and why would the other model fail?

Correct Answer

> Successful Model: Model Beta

> Why it Works: Because it uses **Dense Embeddings**, it recognizes that “Cat” and “Kitten” share a similar “**Statistical Neighborhood**”. This results in a high **Cosine Similarity** score because their vectors point in a similar direction.

> **Why the Other Fails:** Model Alpha (One-Hot) would fail because every word is **Orthogonal** to every other word. The **Dot Product** between Cat and Kitten would be exactly 0, telling the compute they are completely unrelated.

Beyond Static Meaning: The Path to Dynamic Context

The Chapter Milestone

> **Numerical Foundation:** We moved from “Words for Humans” to “Numbers for Computers”

> **The Statistical Era:** N-grams taught us to predict the next word using local frequency and the Markov Assumption.

> **The Geometric Leap:** Dense Embeddings transformed isolated “word islands” into a structured map of human ideas.

REMAINING BARRIER:

> **Static Limitations:** While dense vectors capture “The Company We Keep,” they remain fixed in space once training ends.
> **Contextual Blindness:** A static model can identify “Bank”as a location, but it cannot “see” the surrounding sentence to decide if you are fishing or depositing money.

NEXT CHAPTER: To reach “True Intelligence,” we need a system that doesn’t just store meaning, but **computes it on the fly.**